



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ _____

КАФЕДРА _____ СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ (ИУ5) _____

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА К НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ

НА ТЕМУ:

Анализ данных «Питательность меню McDonald's»

Студент _____ ИУ5-51Б _____
(Группа)

(Подпись, дата) Бондаренко Д. К.
(И.О.Фамилия)

Руководитель

(Подпись, дата) Маслеников К. Ю.
(И.О.Фамилия)

Москва, 2022 г.

Аннотация

В задании необходимо проанализировать базу данных “ Nutrition Facts for McDonald's Menu” и очистить от ненужных данных, также нужно выдвинуть гипотезы, проанализировать их и составить диаграммы по данным гипотезам. В ходе работы будут использоваться такие библиотеки как: «Pandas» и «Seaborn». Анализ будет проводиться с использованием «Jupyter Notebook».

Также будут проводиться корреляционный анализ, агрегирование данных, оптимизация памяти, удаление дубликатов, очистка данных. По имеющимся данным будет производиться анализ гипотез и будут построены соответствующие диаграммы.

В работе будут рассмотрены проблемы питательности меню McDonald's и качества питания клиентов, будут получены выводы, которые могут быть использованы для внесения изменений в меню или для планирования рекламных ходов. И, в конце концов, будет сделан вывод о проделанной работе с данным датасетом.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 Определение данных.....	5
2 Формулирование гипотез	7
3 Загрузка данных в Python	8
4 Проверка и очистка данных	9
5 Преобразование данных	10
6 Выбор данных для анализа.....	12
7 Исследование и визуализация данных.....	15
8 Проверка гипотез.....	22
ЗАКЛЮЧЕНИЕ	25
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	26

ВВЕДЕНИЕ

Анализ меню McDonald's необходим для определения путей улучшения качества питания клиентов и, как следствие, поддержания статуса сети питания.

Цели работы:

- 1) углубление теоретических знаний в сфере анализа и визуализации данных;
- 2) получение навыков разработки систем оперативного анализа данных для информационной системы по некоторой предметной области;
- 3) освоение проведения анализа данных с использованием языка Python;
- 4) исследование некоторых методов и функций обработки и визуализации данных в библиотеках языка Python;
- 5) изучение методов формулировки и анализа гипотез по выбранной предметной области.

Задачи:

- 1) определение данных;
- 2) формулирование гипотез;
- 3) загрузка данных в Python;
- 4) проверка данных;
- 5) очистка данных: устранение ошибок, работа с пропущенными значениями, обработка дубликатов;
- 6) преобразование данных;
- 7) агрегирование и выбор данных для анализа;
- 8) исследование данных;
- 9) визуализация данных;
- 10) подтверждение или опровержение поставленных гипотез;
- 11) формулирование выводов и ограничений.

1 Определение данных

Для проведения анализа данных был выбран набор данных «Питательность меню McDonald's». В наборе содержатся следующие поля:

- 1) Category – категория блюда или напитка,
- 2) Item – наименование блюда или напитка,
- 3) Serving Size – размер порции,
- 4) Calories – количество калорий,
- 5) Calories from Fat – калории с жиров,
- 6) Total Fat – количество жиров,
- 7) Total Fat (% Daily Value) – процент жиров от дневной нормы жиров,
- 8) Saturated Fat – количество насыщенных жиров,
- 9) Saturated Fat (% Daily Value) – процент насыщенных жиров от дневной нормы насыщенных жиров,
- 10) Trans Fat – количество трансжиров,
- 11) Cholesterol – количество холестерина,
- 12) Cholesterol (% Daily Value) – процент холестерина от дневной нормы холестерина,
- 13) Sodium – количество натрия,
- 14) Sodium (% Daily Value) – процент натрия от дневной нормы натрия,
- 15) Carbohydrates – количество углеводов,
- 16) Carbohydrates (% Daily Value) – процент углеводов от дневной нормы углеводов,
- 17) Dietary Fiber – количество пищевых волокон,
- 18) Dietary Fiber (% Daily Value) – процент пищевых волокон от дневной нормы пищевых волокон,
- 19) Sugars – количество сахаров,
- 20) Protein – количество протеина,
- 21) Vitamin A (% Daily Value) – процент витамина А от дневной нормы витамина А,

22) Vitamin C (% Daily Value) – процент витамина С от дневной нормы витамина С,

23) Calcium (% Daily Value) – процент кальция от дневной нормы кальция,

24) Iron (% Daily Value) – процент железа от дневной нормы железа.

Результаты анализа могут быть использованы в следующих целях:

- 1) изменение состава существующих блюд и напитков;
- 2) изменение состава действительного меню McDonald's;
- 3) планирование введения новых блюд в меню McDonald's;
- 4) реклама блюд и напитков;
- 5) составление комбо из блюд и напитков.

2 Формулирование гипотез

Гипотеза 1: «Среднее содержание витаминов в блюдах и напитках любой категории не превышает дневную норму потребления»

Обоснование: Избыток витаминов может быть вреден для организма. Справедливо предположить, что сеть питания McDonald's контролирует содержание витаминов в блюдах и не допускает их избытка.

Гипотеза 2: «В среднем наиболее калорийной категорией меню является категория, связанная с блюдами, а не напитками»

Обоснование: Когда люди голодны, они в первую очередь заказывают еду, а не напитки. Естественным будет предположение, что самой калорийной категорией меню является категория, связанная с едой.

Гипотеза 3: «Среди углеводов сахара преобладают только в категориях напитков и десертов»

Обоснование: Сладкими в меню являются обычно напитки и десерты, в то время как блюда остальных категорий обычно сладкими не являются. Поэтому логичным было бы предположение о преобладании среди углеводов в блюдах тех углеводов, что к сахарам не относятся.

3 Загрузка данных в Python

Для работы с данными в Python импортируем полезные библиотеки [1] [2] [3] [4] [5](Рисунок 3.1). Среди них numpy, pandas, matplotlib, squarify, math, seaborn, scipy и plotly. Некоторые из них используются для создания отдельной диаграммы. Например, squarify импортируется для создания древовидной карты.

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import squarify
from pandas.plotting import parallel_coordinates
import math
import seaborn as sns
import scipy
import plotly
```

Рисунок 3.1 – Импорт библиотек

Загрузим данные из csv файла в датафрейм (Рисунок 3.2). Выведем первые 5 записей датафрейма. Как можно увидеть, данные были загружены корректно, но количество столбцов датафрейма большое. Но это будет исправлено удалением не нужных для анализа столбцов из датафрейма.

```
In [2]: dataset = pd.read_csv('menu.csv')
```

```
In [3]: dataset.head()
```

Out[3]:

	Category	Item	Serving Size	Calories	Calories from Fat	Total Fat	Total Fat (% Daily Value)	Saturated Fat	Saturated Fat (% Daily Value)	Trans Fat	...	Carbohydrates	Carbohydrates (% Daily Value)	Dietary Fiber	Dietary Fiber (% Daily Value)	Sugars	Protein
0	Breakfast	Egg McMuffin	4.8 oz (136 g)	300	120	13.0	20	5.0	25	0.0	...	31	10	4	17	3	
1	Breakfast	Egg White Delight	4.8 oz (135 g)	250	70	8.0	12	3.0	15	0.0	...	30	10	4	17	3	
2	Breakfast	Sausage McMuffin	3.9 oz (111 g)	370	200	23.0	35	8.0	42	0.0	...	29	10	4	17	2	
3	Breakfast	Sausage McMuffin with Egg	5.7 oz (161 g)	450	250	28.0	43	10.0	52	0.0	...	30	10	4	17	2	
4	Breakfast	Sausage McMuffin with Egg Whites	5.7 oz (161 g)	400	210	23.0	35	8.0	42	0.0	...	30	10	4	17	2	

5 rows x 24 columns

Рисунок 3.2 – Загрузка данных

4 Проверка и очистка данных

Произведем удаление столбцов, содержащих второстепенную для анализа информацию. Такими столбцами является большинство столбцов, связанных с дневной нормой содержания элементов, а так же столбцы, связанные с элементами, не имеющими высокого приоритета при анализе питательности пищи (Рисунок 4.1).

```
In [4]: dataset.drop(['Total Fat (% Daily Value)', 'Saturated Fat', 'Saturated Fat (% Daily Value)',
                    'Cholesterol (% Daily Value)', 'Sodium', 'Sodium (% Daily Value)',
                    'Carbohydrates (% Daily Value)', 'Dietary Fiber', 'Dietary Fiber (% Daily Value)',
                    'Calcium (% Daily Value)'], axis=1, inplace=True)

In [5]: dataset.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 260 entries, 0 to 259
Data columns (total 14 columns):
#   Column                                Non-Null Count  Dtype
---  ---                                ---
0   Category                             260 non-null    object
1   Item                                 260 non-null    object
2   Serving Size                         260 non-null    object
3   Calories                             260 non-null    int64
4   Calories from Fat                    260 non-null    int64
5   Total Fat                            260 non-null    float64
6   Trans Fat                           260 non-null    float64
7   Cholesterol                          260 non-null    int64
8   Carbohydrates                       260 non-null    int64
9   Sugars                              260 non-null    int64
10  Protein                             260 non-null    int64
11  Vitamin A (% Daily Value)            260 non-null    int64
12  Vitamin C (% Daily Value)            260 non-null    int64
13  Iron (% Daily Value)                 260 non-null    int64
dtypes: float64(2), int64(9), object(3)
memory usage: 28.6+ KB
```

Рисунок 4.1 – Удаление ненужных столбцов

Проверим наличие пропусков и дубликатов среди данных (Рисунок 4.2). Пропуски и дубликаты отсутствуют. Очистка данных не требуется.

```
In [6]: dataset.isna().sum()

Out[6]: Category                                0
        Item                                    0
        Serving Size                            0
        Calories                                0
        Calories from Fat                       0
        Total Fat                               0
        Trans Fat                              0
        Cholesterol                             0
        Carbohydrates                           0
        Sugars                                  0
        Protein                                 0
        Vitamin A (% Daily Value)               0
        Vitamin C (% Daily Value)               0
        Iron (% Daily Value)                   0
        dtype: int64

In [7]: dataset.duplicated().sum()

Out[7]: 0
```

Рисунок 4.2 – Количество пропусков и дубликатов

5 Преобразование данных

Получим информацию о типах данных столбцов и использовании памяти (Рисунок 5.1).

```
In [8]: dataset.info(memory_usage='deep')

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 260 entries, 0 to 259
Data columns (total 14 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   Category                             260 non-null    object
 1   Item                                 260 non-null    object
 2   Serving Size                         260 non-null    object
 3   Calories                             260 non-null    int64
 4   Calories from Fat                    260 non-null    int64
 5   Total Fat                           260 non-null    float64
 6   Trans Fat                           260 non-null    float64
 7   Cholesterol                         260 non-null    int64
 8   Carbohydrates                       260 non-null    int64
 9   Sugars                              260 non-null    int64
10   Protein                             260 non-null    int64
11   Vitamin A (% Daily Value)           260 non-null    int64
12   Vitamin C (% Daily Value)           260 non-null    int64
13   Iron (% Daily Value)                 260 non-null    int64
dtypes: float64(2), int64(9), object(3)
memory usage: 80.3 KB
```

Рисунок 5.1 – Получение информации о типах данных и памяти

Преобразуем данные для меньшего расхода памяти (Рисунок 5.2). Численные столбцы из float64 и int64 преобразуем в float16 и int16 соответственно. Все столбцы с типом object приведем к строковому типу string. После этих преобразований объем используемой памяти сократился с 80,3 КБ до 63,5 КБ.

```
In [9]: copied_ds = dataset.copy()

for col in dataset.select_dtypes(include=['object']).columns:
    copied_ds[col] = copied_ds[col].astype('string')
for col in dataset.select_dtypes(include=['float64']).columns:
    copied_ds[col] = copied_ds[col].astype('float16')
for col in dataset.select_dtypes(include=['int64']).columns:
    copied_ds[col] = copied_ds[col].astype('int16')

dataset = copied_ds
```

```
In [10]: dataset.info(memory_usage='deep')

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 260 entries, 0 to 259
Data columns (total 14 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   Category                             260 non-null    string
 1   Item                                 260 non-null    string
 2   Serving Size                         260 non-null    string
 3   Calories                             260 non-null    int16
 4   Calories from Fat                    260 non-null    int16
 5   Total Fat                           260 non-null    float16
 6   Trans Fat                           260 non-null    float16
 7   Cholesterol                         260 non-null    int16
 8   Carbohydrates                       260 non-null    int16
 9   Sugars                              260 non-null    int16
10  Protein                             260 non-null    int16
11  Vitamin A (% Daily Value)           260 non-null    int16
12  Vitamin C (% Daily Value)           260 non-null    int16
13  Iron (% Daily Value)                 260 non-null    int16
dtypes: float16(2), int16(9), string(3)
memory usage: 63.5 KB
```

Рисунок 5.2 – Преобразование типов данных

6 Выбор данных для анализа

Далее часто придется сравнивать различные категории блюд и напитков на основе определенных средних показателей, поэтому стоит подготовить заранее данные об этих показателях. Для этого создадим датафрейм с информацией о среднем проценте от дневной нормы содержания витаминов в блюдах и напитках различных категорий путем слияния нескольких датафреймов. Сначала создадим отдельные датафреймы со средним процентом от дневной нормы содержания витаминов в блюдах и напитках различных категорий (Рисунок 6.1).

```
In [11]: vitA=pd.DataFrame(dataset.groupby('Category')['Vitamin A (% Daily Value)'].mean())  
vitC=pd.DataFrame(dataset.groupby('Category')['Vitamin C (% Daily Value)'].mean())
```

```
In [12]: vitA
```

Out[12]:

Vitamin A (% Daily Value)	
Category	
Beef & Pork	6.933333
Beverages	0.740741
Breakfast	6.928571
Chicken & Fish	20.444444
Coffee & Tea	10.736842
Desserts	5.142857
Salads	146.666667
Smoothies & Shakes	18.750000
Snacks & Sides	4.846154

```
In [13]: vitC
```

Out[13]:

Vitamin C (% Daily Value)	
Category	
Beef & Pork	7.333333
Beverages	23.481481
Breakfast	8.904762
Chicken & Fish	12.629630
Coffee & Tea	0.000000
Desserts	4.142857
Salads	28.333333
Smoothies & Shakes	6.964286
Snacks & Sides	28.153846

Рисунок 6.1 – Датафреймы с процентами от дневной нормы витаминов

Затем объединим эти датафреймы в один (Рисунок 6.2).

```
In [14]: vits=vitA.merge(vitC, left_on='Category', right_on='Category',
                        suffixes=('_left', '_right'))
```

```
In [15]: vits
```

```
Out[15]:
```

	Vitamin A (% Daily Value)	Vitamin C (% Daily Value)
Category		
Beef & Pork	6.933333	7.333333
Beverages	0.740741	23.481481
Breakfast	6.928571	8.904762
Chicken & Fish	20.444444	12.629630
Coffee & Tea	10.736842	0.000000
Desserts	5.142857	4.142857
Salads	146.666667	28.333333
Smoothies & Shakes	18.750000	6.964286
Snacks & Sides	4.846154	28.153846

Рисунок 6.2 – Промежуточный объединенный датафрейм

После этого создадим датафрейм, содержащий названия категорий, и объединим его с промежуточным объединенным датафреймом, получив итоговый объединенный датафрейм (Рисунок 6.3).

```
In [16]: d = {'Category': ['Beef & Pork', 'Beverages', 'Breakfast', 'Chicken & Fish', 'Coffee & Tea', 'Desserts', 'Salads',
                        'Smoothies & Shakes', 'Snacks & Sides'], 'Category': ['Beef & Pork', 'Beverages', 'Breakfast',
                        'Smoothies & Shakes', 'Snacks & Sides']}
df = pd.DataFrame(data=d)
df
```

```
Out[16]:
```

	Category
0	Beef & Pork
1	Beverages
2	Breakfast
3	Chicken & Fish
4	Coffee & Tea
5	Desserts
6	Salads
7	Smoothies & Shakes
8	Snacks & Sides

```
In [17]: vits = vits.merge(df, left_on='Category', right_on='Category',
                        suffixes=('_left', '_right'))
```

```
In [18]: vits
```

```
Out[18]:
```

	Category	Vitamin A (% Daily Value)	Vitamin C (% Daily Value)
0	Beef & Pork	6.933333	7.333333
1	Beverages	0.740741	23.481481
2	Breakfast	6.928571	8.904762
3	Chicken & Fish	20.444444	12.629630
4	Coffee & Tea	10.736842	0.000000
5	Desserts	5.142857	4.142857
6	Salads	146.666667	28.333333
7	Smoothies & Shakes	18.750000	6.964286
8	Snacks & Sides	4.846154	28.153846

Рисунок 6.3 – Получение итогового объединенного датафрейма

Также понадобится датафрейм с информацией о среднем содержании сахаров и углеводов в блюдах и напитках различных категорий. Создадим его аналогичным образом (Рисунок 6.4).

```
In [19]: sug=pd.DataFrame(dataset.groupby('Category')['Sugars'].mean())
carb=pd.DataFrame(dataset.groupby('Category')['Carbohydrates'].mean())

In [20]: merged=carb.merge(sug, left_on='Category', right_on='Category',
    suffixes=('_left', '_right'))

In [21]: d = {'Category': ['Beef & Pork', 'Beverages', 'Breakfast', 'Chicken & Fish', 'Coffee & Tea', 'Desserts', 'Salads',
    'Smoothies & Shakes', 'Snacks & Sides'], 'Category': ['Beef & Pork', 'Beverages', 'Breakfast',
    'Chicken & Fish', 'Coffee & Tea', 'Desserts', 'Salads',
    'Smoothies & Shakes', 'Snacks & Sides']}
df = pd.DataFrame(data=d)

In [22]: merged= merged.merge(df, left_on='Category', right_on='Category',
    suffixes=('_left', '_right'))
merged
```

Out[22]:

	Category	Carbohydrates	Sugars
0	Beef & Pork	40.133333	8.800000
1	Beverages	28.814815	27.851852
2	Breakfast	49.761905	8.261905
3	Chicken & Fish	49.074074	7.333333
4	Coffee & Tea	44.526316	39.610526
5	Desserts	34.857143	26.142857
6	Salads	21.666667	6.833333
7	Smoothies & Shakes	90.428571	77.892857
8	Snacks & Sides	29.153846	4.076923

Рисунок 6.4 – Получение датафрейма для сахаров и углеводов

Еще одним полезным датафреймом будет датафрейм с информацией о среднем проценте от дневной нормы содержания железа в блюдах и напитках различных категорий (Рисунок 6.5).

```
In [34]: iron=pd.DataFrame(dataset.groupby('Category')['Iron (% Daily Value)'].mean())

In [35]: d = {'Category': ['Beef & Pork', 'Beverages', 'Breakfast', 'Chicken & Fish', 'Coffee & Tea', 'Desserts', 'Salads',
    'Smoothies & Shakes', 'Snacks & Sides'], 'Category': ['Beef & Pork', 'Beverages', 'Breakfast',
    'Chicken & Fish', 'Coffee & Tea', 'Desserts', 'Salads',
    'Smoothies & Shakes', 'Snacks & Sides']}
df = pd.DataFrame(data=d)

In [36]: iron = iron.merge(df, left_on='Category', right_on='Category',
    suffixes=('_left', '_right'))
```

Рисунок 6.5 – Получение датафрейма для железа

7 Исследование и визуализация данных

Найдем распределение позиций меню по категориям (Рисунок 7.1). Посчитаем количество позиций, относящихся к каждой из категорий, и на основании полученной информации построим круговую диаграмму с процентным соотношением количества позиций каждой категории. Наибольшее количество позиций принадлежит категории «Coffee & Tea», что может быть объяснено тем, что чай и кофе могут подаваться порциями различных объемов, из-за чего количество позиций меню данной категории возрастает.

```
In [23]: types = dataset['Category'].value_counts()
types = pd.DataFrame({'Category':types.index, 'Count':types.values})
plt.pie(types['Count'], labels=types['Category'], autopct = lambda p: format(p, '.0f') if p > 1 else None)
plt.show()
```

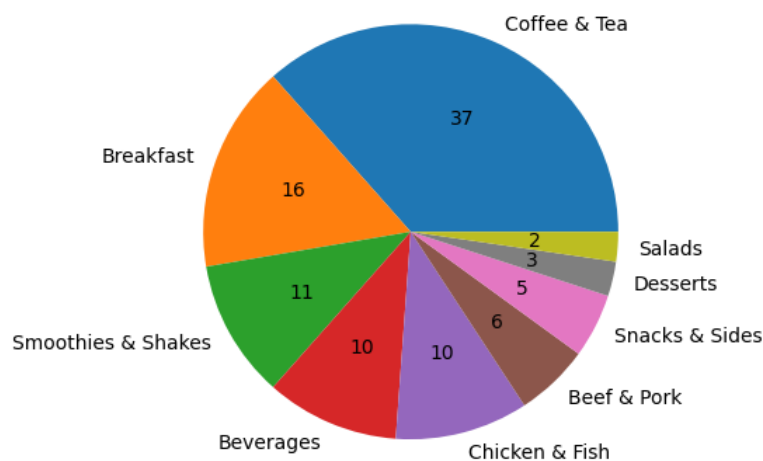


Рисунок 7.1 – Круговая диаграмма распределения позиций меню по категориям

Рассмотрим категорию «Coffee & Tea» в отдельности. Найдем распределение позиций меню, связанных с кофе и чаем, по порциям различного размера (Рисунок 7.2). Посчитаем количество и построим древовидную карту [6]. Как видно из карты, существует множество различных размеров порций, в которых подаются кофе и чай, притом, что сам вид кофе или чая может повторяться. Это подтверждает предположение о преобладании позиций меню, связанных с кофе и чаем.

```
In [24]: data = dataset.loc[dataset['Category'] == 'Coffee & Tea']
data = data['Serving Size'].value_counts()
squarify.plot(sizes=data.values, label=data.index, alpha=.8 )
plt.axis('off')
plt.show()
```

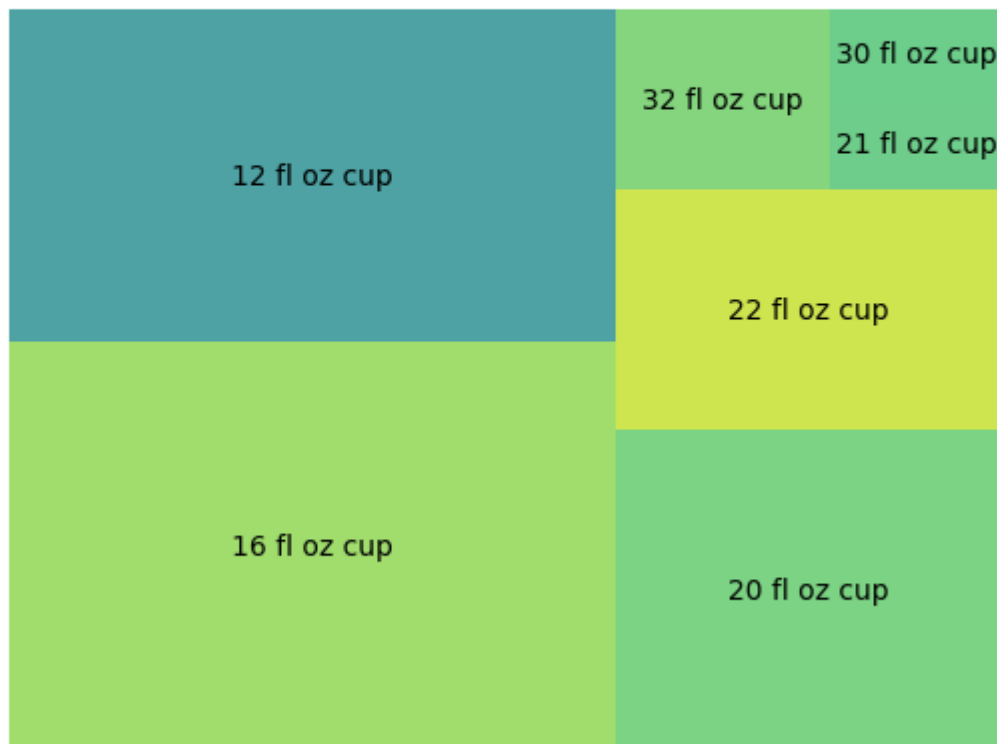


Рисунок 7.2 – Древовидная карта количества позиций по размерам порций

Построим диаграмму разброса позиций меню по содержанию в них холестерина и трансжиров, являющихся для организма вредными (Рисунок 7.3). Каждая точка диаграммы окрашена в цвет, соответствующий определенной категории блюда или напитка. Таким образом, была получена импровизированная система координат, связанная с содержанием вредных веществ в блюдах и напитках: чем выше и правее лежит точка, тем больше вреда может принести соответствующее блюдо или напиток. Можно увидеть, что блюда категории «Breakfast» содержат намного больше холестерина, чем блюда других категорий, а у блюд категории «Beef & Pork» аналогичная ситуация с трансжирами.


```
In [25]: sns.lmplot( x='Trans Fat', y='Cholesterol', data=dataset, fit_reg=False, hue='Category', legend=False)

# Move the Legend to an empty part of the plot
plt.legend(loc='upper right', fontsize=8)

plt.show()
```

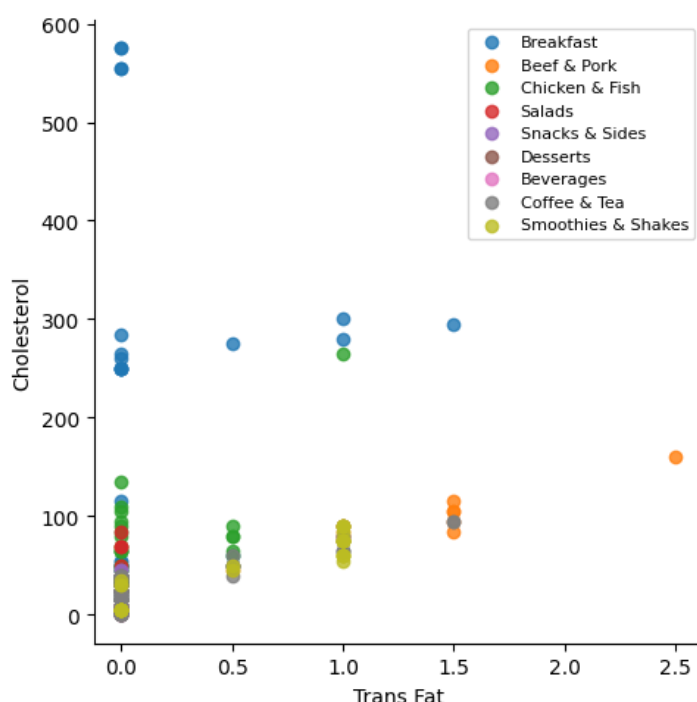


Рисунок 7.3 – Диаграмма разброса позиций по холестерину и трансжирам

Узнаем среднее содержание жиров в блюдах и напитках различных категорий. Рассчитаем заданный показатель и построим столбчатую диаграмму (Рисунок 7.4). Диаграмма показывает, что блюда категорий «Chicken & Fish» и «Breakfast» в среднем содержат больше жиров, чем блюда категории «Beef & Pork», хотя предыдущая диаграмма и предоставила информацию о том, что категория «Beef & Pork» наиболее богата трансжирами. Таким образом, из последних двух диаграмм можно сделать вывод, что блюдам категории «Chicken & Fish» соответствует наилучшее соотношение трансжиров к жирам, а также наименьшее количество холестерина, в то время, как блюда двух других категорий отстают по этим показателям.

```
In [26]: height = dataset.groupby('Category')['Total Fat'].mean()
bars = ('Beef & Pork', 'Beverages', 'Breakfast', 'Chicken & Fish', 'Coffee & Tea', 'Desserts',
        'Salads', 'Smoothies & Shakes', 'Snacks & Sides')

y_pos = np.arange(len(bars))

# Create bars
plt.barh(y_pos, height)

# Create names on the x-axis
plt.yticks(y_pos, bars)

# Show graphic
plt.show()
```

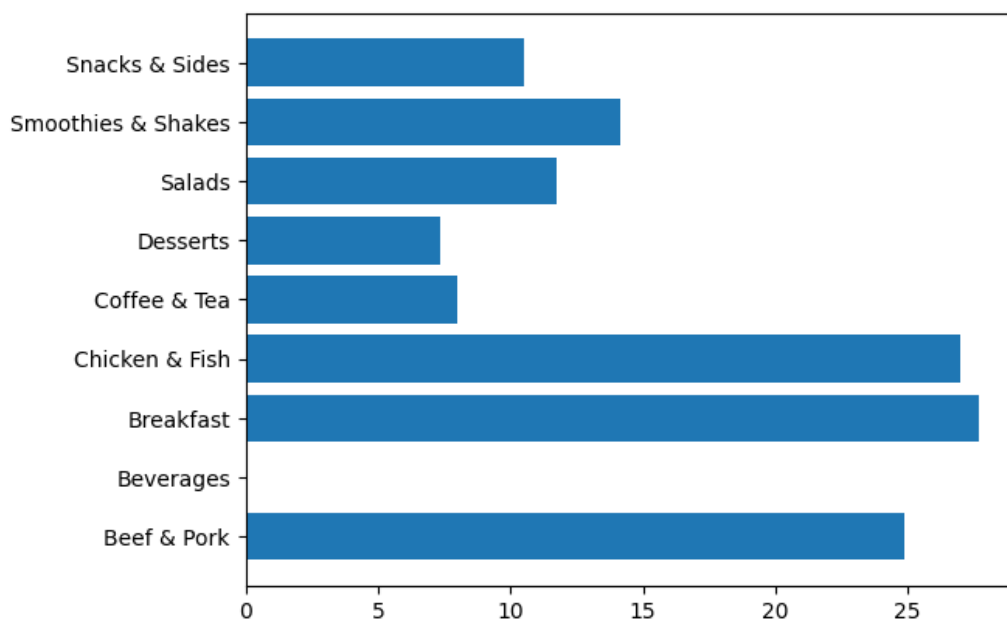


Рисунок 7.4 – Диаграмма среднего количества жиров по категориям

Построим матрицу корреляций показателей питательности блюд и напитков меню McDonald's (Рисунок 7.5). Сильная положительная корреляция наблюдается между содержанием в блюдах и напитках калорий и содержанием в них жиров, содержанием калорий и содержанием углеводов (так как жиры и углеводы являются источниками калорий), содержанием сахаров и содержанием углеводов (так как сахара являются углеводами), содержанием протеина и содержанием жиров, содержанием протеина и содержанием железа. Также можно заметить, что существует отрицательная корреляция между содержанием сахаров в блюдах и напитках и содержанием в них железа.

```
In [42]: plt.figure(figsize = (14,8))

m = np.triu(np.ones_like(dataset.corr(), dtype=bool))

sns.heatmap(dataset.corr(), mask = m, annot = True, vmin=- 1.0, vmax= 1.0, center = 0, cmap = 'RdBu_r');
```

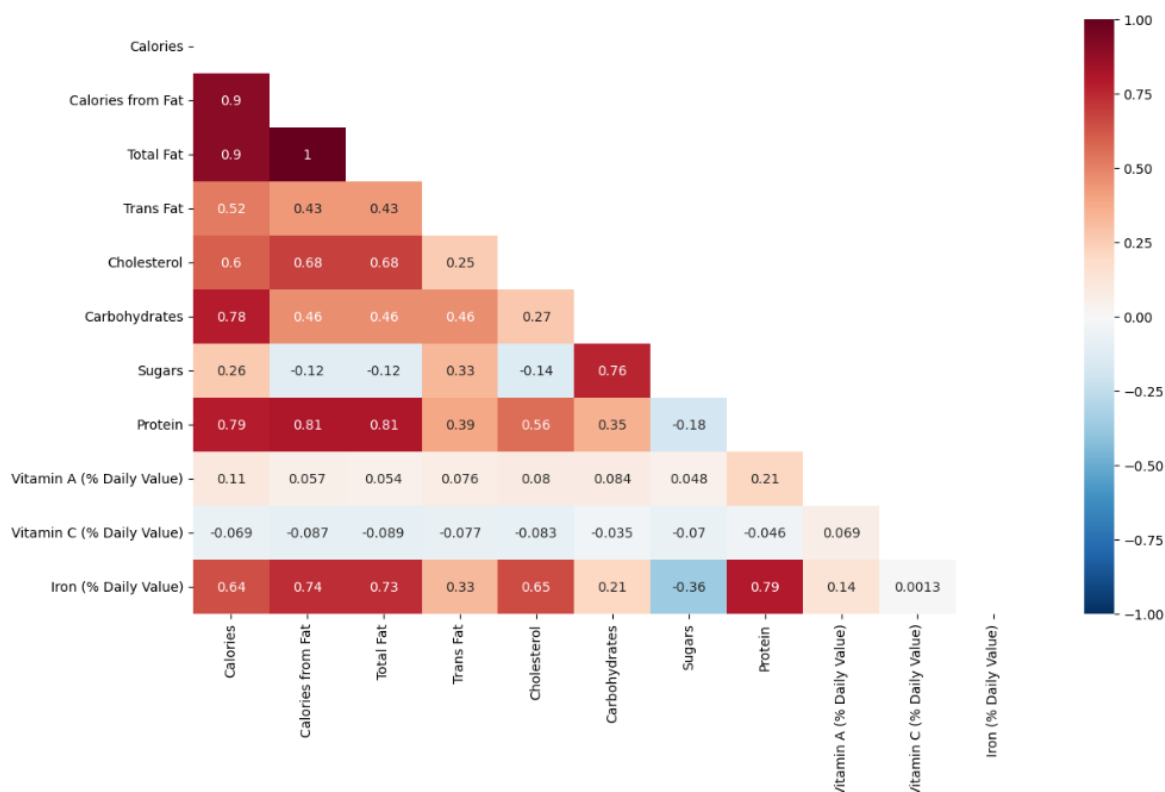


Рисунок 7.5 – Матрица корреляций показателей питательности

Рассмотрим распределение содержания протеина в блюдах и напитках различных категорий. Для этого построим блочную диаграмму, дающую представление о характере распределения для каждой категории блюд и напитков (Рисунок 7.6). Диаграмма показывает, что содержание протеина в напитках категории «Beverages» распределено крайне неравномерно. Это объясняется тем, что большинство напитков не содержит протеин. Поэтому минимум и первый квартиль соответствующей диаграммы слились на уровне 0. Также можно увидеть отчетливый выброс распределения содержания протеина у блюд категории «Chicken & Fish».

```
In [30]: sns.boxplot(x='Category', y='Protein', data=dataset)
ax = sns.stripplot(x='Category', y='Protein', data=dataset, color="orange", jitter=0.2, size=2.5)
plt.xticks(rotation=90)
plt.show()
```

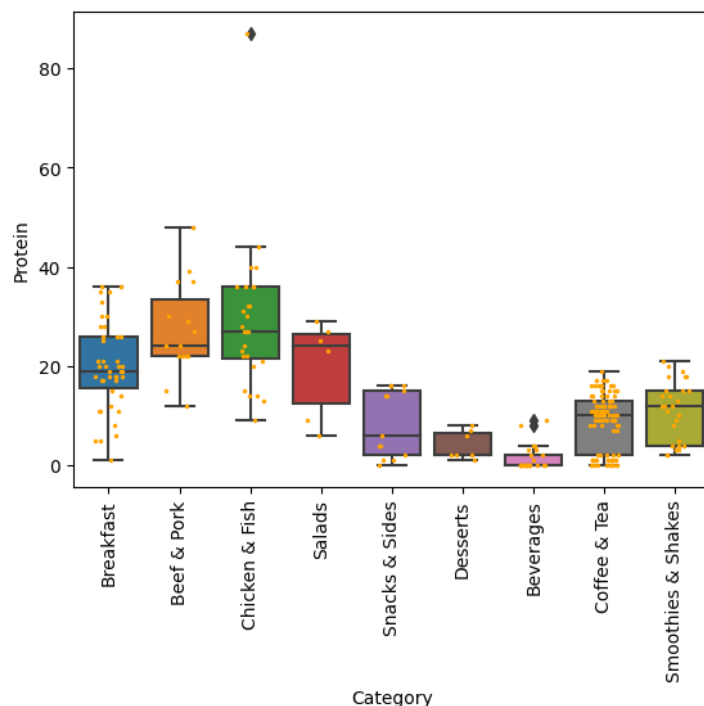


Рисунок 7.6 – Блочная диаграмма распределения протеина по категориям

Этому выбросу соответствует блюдо, содержащее в себе 40 куриных наггетсов, что объясняет большое содержание протеина в этом блюде (Рисунок 7.7).

```
In [31]: sorted_df = dataset.sort_values('Protein', ascending=False)
sorted_df
```

Out[31]:

	Category	Item	Serving Size	Calories	Calories from Fat	Total Fat	Trans Fat	Cholesterol	Carbohydrates	Sugars	Protein	Vitamin A (% Daily Value)	Vitamin C (% Daily Value)	Iron (% Daily Value)
82	Chicken & Fish	Chicken McNuggets (40 piece)	22.8 oz (646 g)	1880	1060	118.0	1.0	265	118	1	87	0	15	25
47	Beef & Pork	Double Quarter Pounder with Cheese	10 oz (283 g)	750	380	43.0	2.5	160	42	10	48	10	2	35
81	Chicken & Fish	Chicken McNuggets (20 piece)	11.4 oz (323 g)	940	530	59.0	0.0	135	59	0	44	0	8	10
60	Chicken & Fish	Premium Grilled Chicken Club Sandwich	8.3 oz (235 g)	510	180	20.0	0.0	105	44	9	40	8	20	20
64	Chicken & Fish	Bacon Clubhouse Grilled Chicken Sandwich	9.5 oz (270 g)	590	230	25.0	0.0	110	51	14	40	8	30	15
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
110	Beverages	Coca-Cola Classic (Small)	16 fl oz cup	140	0	0.0	0.0	0	39	39	0	0	0	0
136	Beverages	Dasani Water Bottle	16.9 fl oz	0	0	0.0	0.0	0	0	0	0	0	0	0
127	Beverages	Sprite (Medium)	21 fl oz cup	200	0	0.0	0.0	0	54	54	0	0	0	0
138	Coffee & Tea	Iced Tea (Medium)	21 fl oz cup	0	0	0.0	0.0	0	0	0	0	0	0	0
137	Coffee & Tea	Iced Tea (Small)	16 fl oz cup	0	0	0.0	0.0	0	0	0	0	0	0	0

Рисунок 7.7 – Датафрейм, отсортированный по содержанию протеина в блюде

Узнаем средний процент от дневной нормы питания содержания в блюдах и напитках различных категорий железа. Построим на основе созданного ранее датафрейма лоллипор-диаграмму (Рисунок 7.8). Данная диаграмма схожа с рассмотренной ранее блочной диаграммой распределения протеина. Наивысшие показатели также имеют блюда категорий «Beef & Pork», «Breakfast» и «Chicken & Fish», а наименьшего показателя достигают напитки категории «Beverages». Прослеживается связь между этими показателями питательности, на которую указывает большой коэффициент корреляции из матрицы корреляций показателей питательности.

```
In [37]: ordered_df = iron.sort_values(by='Iron (% Daily Value)')
my_range=range(1,len(iron.index)+1)

plt.hlines(y=my_range, xmin=0, xmax=ordered_df['Iron (% Daily Value)'], color='skyblue')
plt.plot(ordered_df['Iron (% Daily Value)'], my_range, "D")
plt.yticks(my_range, ordered_df['Category'])
plt.show()
```

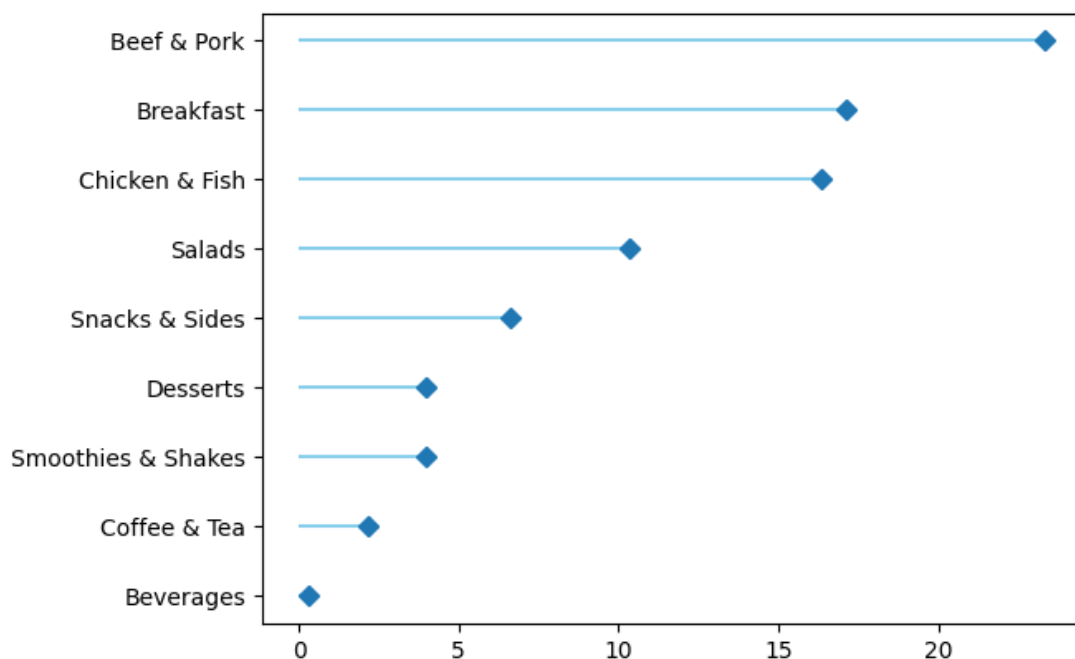


Рисунок 7.8 – Средний процент от дневной нормы питания содержания железа

8 Проверка гипотез

Проверка гипотезы 1: «Среднее содержание витаминов в блюдах и напитках любой категории не превышает дневную норму потребления»

С использованием созданного ранее датафрейма, содержащего информацию о содержании витаминов в блюдах и напитках различных категорий, построим диаграмму разброса категорий блюд и напитков по содержанию в них витаминов (Рисунок 8.1.1). Блюда и напитки ни одной из категорий в среднем не содержат более чем дневную норму витамина С. Но блюда категории «Salads» в среднем превышают даже 140% дневной нормы витамина А.

```
In [28]: sns.lmplot(x='Vitamin A (% Daily Value)', y='Vitamin C (% Daily Value)', data=vits, fit_reg=False, hue='Category', legend=False)

# Move the legend to an empty part of the plot
plt.legend(loc='lower right', fontsize=8)

plt.show()
```

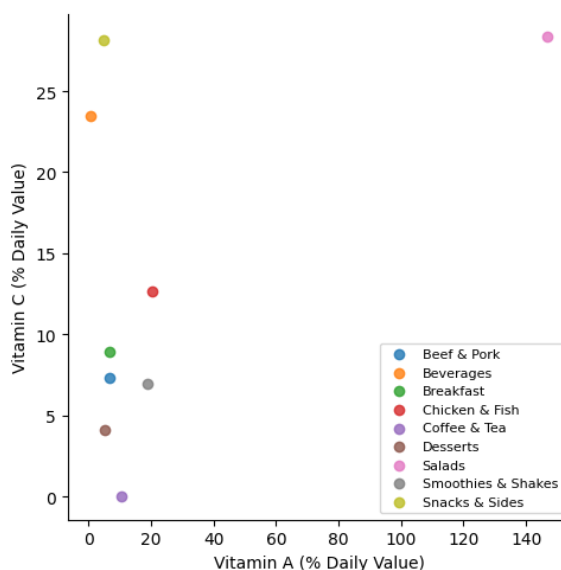


Рисунок 8.1.1 – Диаграмма разброса категорий по содержанию витаминов

Вывод: гипотеза опровергнута. Действительно, витамин А содержится во многих продуктах, используемых в салатах, причем эти салаты часто готовят без термической обработки продуктов, что позволяет им сохранять высокое содержание витаминов. Кроме того, салаты редко заказываются в таких сетях питания, как McDonald's, а остальные блюда не содержат такого количества витамина А, поэтому такое содержание витамина А в салатах McDonald's допустимо.

Проверка гипотезы 2: «В среднем наиболее калорийной категорией меню является категория, связанная с блюдами, а не напитками»

Рассчитаем среднюю калорийность блюд и напитков каждой категории и построим столбчатую диаграмму, столбцы которой будут отсортированы по возрастанию (Рисунок 8.2.1). Самый высокий столбец соответствует категории «Chicken & Fish», эта категория связана с блюдами, а не напитками. Вторым по высоте и почти не отстающим от первого является столбец, соответствующий категории «Smoothies & Shakes», связанной с напитками.

```
In [27]: cal = pd.DataFrame(dataset.groupby('Category')['Calories'].mean()).sort_values('Calories')
plot = cal.plot.bar(y='Calories', color=['b','c','g','r','maroon'], label='Калории')
```

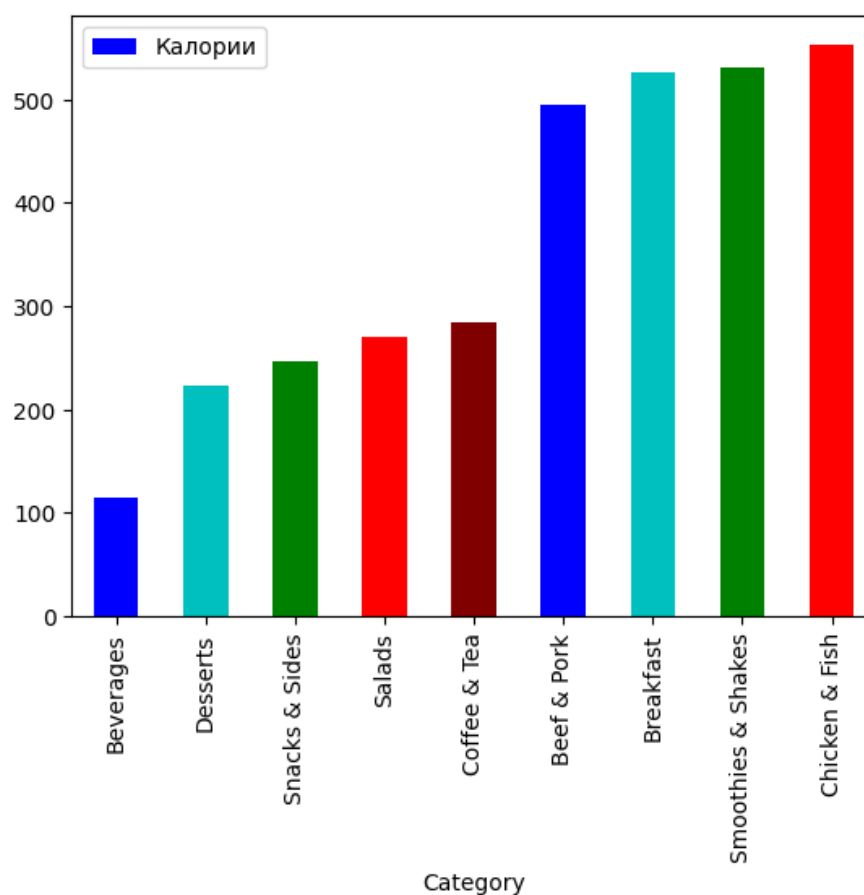


Рисунок 8.2.1 – Диаграмма средней калорийности по категориям

Вывод: гипотеза подтверждена. Но оказалось, что обоснование гипотезы не имеет надежного фундамента, так как одна из категорий, связанная с напитками, почти не уступает первой категории, связанной с блюдами. Смузи и коктейли являются высококалорийными напитками.

Проверка гипотезы 3: «Среди углеводов сахара преобладают только в категориях напитков и десертов»

Для проверки гипотезы построим на основе созданного ранее датафрейма, содержащего информацию о среднем содержании в блюдах различных категорий сахаров и углеводов, диаграмму с параллельными координатами (Рисунок 8.3.1). Так как сахара являются углеводами, нижний конец линий относится к сахарам, а верхний – к углеводам. Чем больше спад, тем меньшую часть углеводов составляют сахара. Из диаграммы видно, что спад менее чем вдвое наблюдается только у категорий «Beverages», «Coffee & Tea», «Desserts» и «Smoothies & Shakes», которые связаны с напитками и десертами.

```
In [33]: parallel_coordinates(merged, 'Category', colormap=plt.get_cmap("Set1"))  
#plt.xticks(rotation=90)  
plt.show()
```

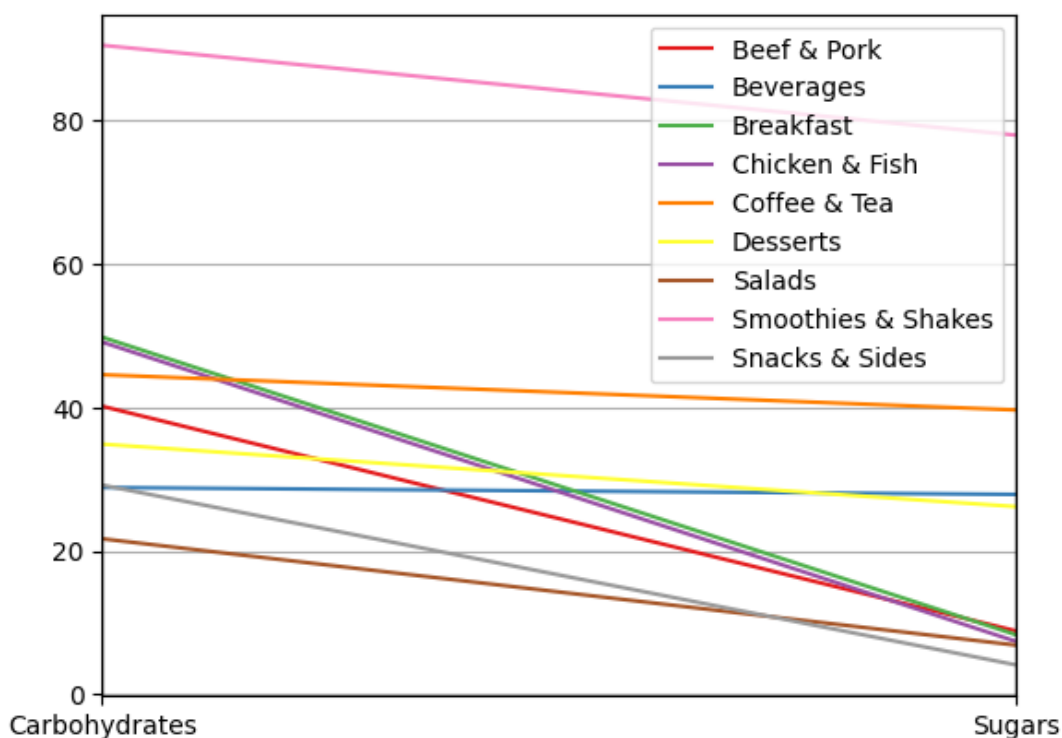


Рисунок 8.3.1 – Диаграмма среднего содержания углеводов и сахаров

Вывод: гипотеза подтверждена. Действительно, в блюда других категорий не добавляют такое количество сахара или других подсластителей, относящихся к сахарам, поэтому в них преобладают другие углеводы.

ЗАКЛЮЧЕНИЕ

В результате научно-исследовательской работы была спроектирована система анализа данных по теме «Питательность меню McDonald's», которая позволяет оценить питательные качества меню McDonald's для дальнейшего повышения качества питания клиентов [1].

В процессе выполнения работы были углублены теоретические знания в сфере анализа и визуализации данных, были получены навыки разработки систем оперативного анализа данных для информационной системы по некоторой предметной области, было освоено проведение анализа данных с использованием языка Python, были исследованы некоторые методы и функции обработки и визуализации данных в библиотеках языка Python [2] [5].

Были выдвинуты и проверены 3 гипотезы, а также было проведено общее исследование данных предметной области, результаты которого могут быть использованы для изменения состава меню и самих блюд и напитков McDonald's, а также в целях создания информационной основы для рекламы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Плас Дж. Вандер. Python для сложных задач: наука о данных и машинное обучение. – СПб.: Питер, 2018. – 576 с.: ил. – (Серия «Бестселлеры O'Reilly»)
2. Официальная документация Python: [Электронный ресурс]. // URL: <https://www.python.org/doc/> (Дата обращения: 09.12.2022)
3. Официальная документация Pandas: [Электронный ресурс]. // URL: <https://pandas.pydata.org/pandas-docs/stable/reference/index.html> (Дата обращения: 09.12.2022)
4. Официальная документация Matplotlib: [Электронный ресурс]. // URL: <https://matplotlib.org/stable/index.html> (Дата обращения: 09.12.2022)
5. Официальная документация Seaborn: [Электронный ресурс]. // URL: <https://seaborn.pydata.org/> (Дата обращения: 09.12.2022)
6. ThePythonGraphGallery: [Электронный ресурс]. // URL: <https://www.python-graph-gallery.com/> (Дата обращения: 09.12.2022)